

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to identify and describe each of the ten entries in the OWASP Web Application Top 10 (2021 edition) and explain the risk each represents in a cloud context.
- By the end of this lecture, you will be able to identify and describe the ten entries in the OWASP API Security Top 10 (2023 edition) and explain why the API risk profile differs from web application risk.
- By the end of this lecture, you will be able to apply OWASP Top 10 categories to evaluate the risk posture of a cloud application under review.
- By the end of this lecture, you will be able to explain how specific OWASP entries such as SSRF and Vulnerable Components are particularly significant in cloud environments.
- By the end of this lecture, you will be able to use OWASP lists as operational tools for penetration testing scope, risk assessment, and communication with development teams.

Introduction

The OWASP Top 10 lists are among the most consequential artefacts in applied application security. They are not academic rankings — they are distilled from real-world vulnerability data aggregated across thousands of organisations and applications, curated by practitioners who track where attacks actually happen and where defences most often fail. For the CCSP exam and for professional practice, the OWASP Web Application Top 10 and the OWASP API Security Top 10 function as operational intelligence: they tell security professionals where to look, what to test, and how to frame risk conversations with development teams and executive stakeholders.

The 2021 edition of the OWASP Web Application Top 10 reflects a significant evolution from earlier versions. It consolidated some long-standing vulnerability categories, elevated Insecure Design to its own entry for the first time, and added Server-Side Request Forgery — a category with particular relevance to cloud environments where internal metadata services are reachable via HTTP. These changes signal shifts in where the security community observes concentrated real-world risk, and understanding why each entry made the list is as important as knowing what it contains.

The OWASP API Security Top 10, updated in 2023, addresses a distinct risk profile. APIs are the connective tissue of cloud applications — every integration, every microservice, every mobile client communicates through them. But API security failures are often missed by assessments designed for traditional web applications. Broken Object Level Authorisation, the top API security risk, is absent from the web top 10 because the mechanics of API interaction create this vulnerability pattern in ways that HTML-based web applications do not. Knowing both lists and understanding where they diverge is central to comprehensive cloud application security coverage.

Core Concepts Explained

OWASP Web Top 10: Broken Access Control (A01) and Cryptographic Failures (A02)

Definition: Broken Access Control (A01) is the most common web application vulnerability found in practice. It occurs when applications fail to enforce what authenticated users are authorised to do — allowing users to access other users' data, perform privileged actions, or bypass intended restrictions. Cryptographic Failures (A02) covers misuse or absence of encryption to protect data in transit and at rest, including weak algorithms, missing encryption, and improper key management.

Business relevance: Broken access control is the leading cause of data exposure in web applications. Cryptographic failures expose data even when access controls are correctly implemented — if an attacker can read encrypted data because the cipher is weak or keys are mismanaged, the access control layer is irrelevant. Both categories represent systemic failures rather than one-off coding errors.

Cloud Security Practitioner relevance: Cloud Security Practitioners should verify access control is enforced at every layer (application, API, data store) and that HTTPS is enforced across all endpoints. Cryptographic control reviews should include cipher suite configuration, certificate management, and data-at-rest encryption for all sensitive data stores.

Real-world example: A cloud-hosted HR application allows employees to view their own payroll records through a URL pattern of `/payroll/{employeeID}`. Security testing reveals that changing the employee ID in the URL to any other valid ID returns that employee's payroll data, with no server-side check that the authenticated user is the employee whose record is being requested. This is a textbook broken access control failure. The fix is a server-side authorisation check that compares the requested employee ID against the authenticated user's identity before returning any data. For cryptographic failures, a parallel finding in the same application reveals that historical payroll data exports stored in cloud object storage use a deprecated AES-128-ECB cipher — both the key length and the cipher mode (which does not provide semantic security) are inadequate.

Risks or limitations: Access control failures are architectural: a URL structure or an API design that exposes resource identifiers without enforcing ownership is a structural flaw that requires redesign, not patching. Cryptographic failures often persist because developers do not recognise that an algorithm in use is deprecated — cipher suite reviews should be a standing component of application security reviews.

OWASP Web Top 10: Injection (A03), Insecure Design (A04), and Security Misconfiguration (A05)

Definition: Injection (A03) covers attacks where untrusted data is sent to an interpreter as part of a command or query, including SQL injection, OS command injection, and LDAP injection. Insecure Design (A04) is a category for structural security failures baked into an application's architecture before a line of code is written. Security Misconfiguration (A05) covers default settings left unchanged, excessive permissions, verbose error messages, and cloud service configurations that expand the attack surface unnecessarily.

Business relevance: SQL injection has been exploitable for over two decades and remains one of the most reliably exploited vulnerabilities because it enables direct database access. Insecure design signals that testing and patching cannot compensate for architectural flaws — the application must be redesigned. Security misconfiguration is the most prevalent cloud vulnerability category because cloud environments have larger, more dynamic configuration surfaces than traditional infrastructure.

Cloud Security Practitioner relevance: Cloud Security Practitioners should verify parameterised queries are used throughout database interaction code, participate in design reviews to catch insecure design patterns before development starts, and use Cloud Security Posture Management (CSPM) tools to detect and alert on configuration drift.

Real-world example: A cloud application accepts a product search term from users and constructs a database query by concatenating the input directly into a SQL string. A penetration tester submits the input ' OR '1'='1 and receives the entire product table in the response. A more targeted payload extracts the user table including password hashes. This is a classic SQL injection vulnerability. The remediation is parameterised queries or an ORM framework that handles input separation from the query structure. For insecure design, a separate review of the same application finds that the password reset flow does not require knowledge of the old password — any authenticated user can reset any other user's password by navigating to a specific URL. This is a design flaw: no amount of code patching fixes it without redesigning the flow. For misconfiguration, a CSPM scan reveals the application's S3 bucket storing user-uploaded documents is publicly readable — a default that was never changed from the initial prototype.

Risks or limitations: Insecure design is the hardest OWASP category to remediate because it requires architectural changes, not code fixes. The best practice is threat modelling and security design review at the architecture phase, before development investment makes changes costly.

OWASP Web Top 10: Vulnerable Components (A06), Authentication Failures (A07), and Integrity Failures (A08)

Definition: Vulnerable and Outdated Components (A06) covers risk from incorporating libraries, frameworks, or other components with known vulnerabilities. Identification and Authentication Failures (A07) covers weaknesses in how applications verify user identity and manage sessions. Software and Data Integrity Failures (A08) covers scenarios where code, updates, or data arrive from untrusted sources without integrity verification — including CI/CD pipeline attacks and insecure deserialization.

Business relevance: Log4Shell demonstrated at scale what A06 means in practice: a single vulnerable library in thousands of applications created a global emergency. A07 failures enable account takeover — the gateway to everything the compromised account can access. A08 integrity failures are increasingly significant as software supply chain attacks target the build and distribution pipeline rather than the application itself.

Cloud Security Practitioner relevance: Cloud Security Practitioners should require SCA in CI/CD pipelines, enforce MFA for all user-facing applications, review CI/CD pipeline security including build artefact signing and supply chain controls, and ensure that deserialization of untrusted data is explicitly prevented or sandboxed.

Real-world example: For A06: a cloud application uses a third-party XML parsing library that has not been updated in three years. A newly disclosed vulnerability in the library allows XML External Entity (XXE) injection — an attacker can use the application's XML parser to read arbitrary files from the server. The finding surfaces through a Software Composition Analysis scan that was not previously part of the pipeline. For A07: a customer authentication endpoint has no protection against credential stuffing — a script can attempt thousands of username/password combinations per minute without triggering a lockout. Adding rate limiting, CAPTCHA challenges after repeated failures, and MFA removes the practical viability of this attack. For A08: a CI/CD pipeline pulls build dependencies directly from a public package registry without verifying checksums or signatures. An attacker who compromises the registry account for a widely used package can push a malicious version that executes during the build process — a supply chain attack that injects malicious code before the application is ever deployed.

Risks or limitations: A08 integrity failures are particularly dangerous because they can compromise applications before they are deployed, making post-deployment security testing insufficient as a standalone control. Build pipeline security is an application security concern.

OWASP Web Top 10: Logging Failures (A09) and SSRF (A10)

Definition: Security Logging and Monitoring Failures (A09) covers inadequate logging of security-relevant events, insufficient monitoring of those logs, and absence of alerting on anomalous patterns. Server-Side Request Forgery (A10) occurs when a web application fetches a remote resource based on user-supplied input and an attacker manipulates that input to cause the server to make requests to internal resources.

Business relevance: A09 failures extend attacker dwell time — the median time from breach to detection remains measured in weeks because most organisations cannot detect malicious activity from application logs that were not designed with security in mind. SSRF entered the OWASP Top 10 explicitly because of cloud environments: the EC2 metadata service, available at a well-known internal IP, is a rich target for SSRF attacks that can yield temporary credentials with significant permissions.

Cloud Security Practitioner relevance: Cloud Security Practitioners should define security logging standards for all cloud applications, implement SIEM integration and alerting on defined anomaly patterns, and review all user-controllable URL fetch functionality in applications for SSRF controls — including disabling redirects, validating URLs against an allowlist, and blocking access to cloud metadata service IP ranges.

Real-world example: For A09: a cloud SaaS application has detailed application error logs but no security event logging — failed authentication attempts, privilege escalation actions, and bulk data exports are not logged as distinct events. During a breach investigation, the team cannot determine when attacker access began or which data was accessed. Implementing structured security logging with a SIEM and alerting on authentication anomalies would have reduced detection time from weeks to hours. For SSRF: a cloud application allows users to submit a URL for the application to fetch and display a preview. An attacker submits `http://169.254.169.254/latest/meta-data/iam/security-credentials/` — the AWS EC2 metadata endpoint. The application fetches it faithfully and displays IAM credentials with a fifteen-minute lifetime. The attacker captures the credentials and uses them before they expire. Mitigation includes validating submitted URLs against an allowlist, blocking private and link-local IP ranges, and implementing IMDSv2 on all EC2 instances (which requires a PUT request before credentials are available, preventing simple GET-based metadata access).

Risks or limitations: SSRF in cloud environments is more dangerous than in on-premises environments because metadata services provide working credentials, not just configuration data. Migrating EC2 instances to IMDSv2 and enforcing this at the AWS account policy level is a best practice that significantly raises the bar for SSRF exploitation.

OWASP API Security Top 10: Key Differences from Web Top 10

Definition: The OWASP API Security Top 10 (2023) addresses risks specific to API-driven architectures. The top two entries — Broken Object Level Authorisation (BOLA) and Broken Authentication — reflect that APIs expose resource identifiers and function endpoints directly to callers in ways HTML web applications do not, creating distinct attack surfaces. Other notable API-specific entries include Unrestricted Resource Consumption (rate limiting failures), Broken Function Level Authorisation (admin functions accessible to standard users), and Unsafe Consumption of Third-Party APIs (trusting external API data without validation).

Business relevance: APIs are the primary interface layer in cloud-native applications. BOLA alone has been implicated in numerous high-profile data exposure incidents because API design patterns that expose resource identifiers (user IDs, account numbers, record IDs) in URLs and request bodies make it trivial to attempt horizontal privilege escalation. Standard web application penetration tests often miss BOLA because they focus on authentication bypass rather than object-level access.

Cloud Security Practitioner relevance: Cloud Security Practitioners should require API security testing that specifically targets BOLA and Broken Function Level Authorisation, not just authentication controls. API

inventories should be maintained to prevent shadow APIs — undocumented endpoints that receive no security governance — from accumulating over time.

Real-world example: A healthcare API exposes patient consultation records at GET /consultations/{consultationID}. The endpoint verifies that the caller holds a valid JWT but does not verify that the patient associated with that consultation is the patient whose account issued the token. A malicious user iterates consultationIDs sequentially and retrieves records for thousands of patients. This is BOLA — the most common API security vulnerability. The fix is an authorisation check within the endpoint that retrieves the patient associated with the consultationID and compares it to the identity in the JWT before returning data. For Broken Function Level Authorisation: the same API exposes DELETE /consultations/{consultationID} which was intended only for administrative users but has no function-level authorisation check — any authenticated user can delete records. Authentication verifies identity; function-level authorisation must separately verify that the identity has the right to invoke the specific function.

Risks or limitations: Improper Inventory Management (API9 in the OWASP API list) is a particularly insidious risk: shadow APIs — endpoints deployed for testing or integration that are never formally decommissioned — accumulate over time with no security governance. Regular API discovery scanning and an enforced API inventory process prevent this accumulation.

Practical Applications

The following scenarios illustrate how a Cloud Security Practitioner uses OWASP lists as operational tools, not reference material, in professional security work.

Using the OWASP Web Top 10 to Structure a Pre-Launch Application Security Review

A Cloud Security Practitioner is asked to conduct a security review of a new cloud-native customer portal before it goes live. Rather than conducting an unstructured assessment, the practitioner uses the OWASP Web Top 10 as a structured review framework. For A01, they confirm that every endpoint returns data only to the user who owns it, with authorisation checks verified through test cases in the DAST report. For A02, they check that all endpoints use TLS 1.2 or higher and that data at rest in cloud storage is encrypted with AES-256. For A03, they review the codebase for any string concatenation in database queries and confirm parameterised queries are used throughout. The structured approach ensures the review is comprehensive and consistently applied, with each Top 10 category documented as either addressed or flagged for remediation. Three findings are raised before launch; all are remediated within the agreed timeline.

Investigating an API Data Exposure Through a BOLA Vulnerability

A Cloud Security Practitioner receives a bug bounty report claiming that the organisation's customer API exposes other users' account data. The practitioner reviews the API specification and identifies that the /account/{accountID}/transactions endpoint accepts an account ID in the path. Testing confirms that an authenticated user can substitute any account ID and receive that account's transaction history — the endpoint verifies authentication but not object-level authorisation. The practitioner escalates to a P1 incident, takes the endpoint offline pending a fix, and works with the development team to implement a server-side check that retrieves the account associated with the authenticated identity and validates that the requested accountID belongs to it. The fix is tested through both a unit test and a targeted DAST case before the endpoint is re-enabled. The practitioner also requests an API-wide review for BOLA patterns on all endpoints that accept resource identifiers, as BOLA is almost never an isolated finding.

Building an API Security Test Plan from the OWASP API Top 10

A Cloud Security Practitioner is asked to define the scope of an API penetration test for a new microservices platform with thirty-two endpoints across eight services. Rather than leaving scope definition to the penetration testing vendor, the practitioner builds a structured test plan mapped to the OWASP API Security Top 10. The plan specifies that every endpoint must be tested for BOLA (can callers access other users' objects by manipulating identifiers), every authentication flow must be tested for token replay and credential stuffing resistance, and every administrative function must be tested for function-level authorisation bypass. Rate limiting is tested through automated request flooding against all public endpoints. Third-party API consumption is reviewed for missing validation of returned data. The structured scope ensures the test covers the risks the OWASP list identifies as most prevalent, rather than focusing only on what the vendor finds most convenient to test.

Best Practices

- **Map every application security assessment against both the OWASP Web Top 10 and the OWASP API Top 10, not just whichever applies to the primary interface.** Many cloud applications expose both a web interface and an API. Assessing only the web interface or only the API misses the risk profile of the other. Create a dual-mapping checklist that confirms coverage of both lists for every application that exposes either interface, and document which test methods (DAST, penetration test, code review) addressed each category.
- **Treat SSRF as a mandatory test case for any cloud application that fetches external URLs or makes server-side HTTP requests.** In cloud environments, SSRF is particularly dangerous because metadata services provide working credentials to an attacker who successfully triggers a server-side fetch to the metadata IP range. All user-controllable URL parameters should be tested for SSRF. Mitigation should include URL allowlisting, blocking of private and link-local ranges, disabling of HTTP redirects, and enforcement of IMDSv2 on all EC2 instances.
- **Test specifically for BOLA on every API endpoint that accepts a resource identifier, as automated scanners often miss this class of vulnerability.** BOLA is the most prevalent API security vulnerability and the most commonly missed by automated scanning tools because it requires understanding the relationship between an authenticated identity and the resource being requested. Manual test cases that substitute one authenticated user's resource identifiers with another's should be a required component of every API penetration test.
- **Use OWASP category language when communicating application security findings to development teams and risk owners.** Framing a finding as 'Broken Object Level Authorisation (OWASP API Top 10 — API1:2023)' communicates severity, provides a reference to remediation guidance, and situates the finding in a shared vocabulary. Development teams who are familiar with OWASP lists can immediately understand the risk class and its remediation options without requiring extensive additional explanation. This makes security findings more actionable and remediation faster.
- **Include the OWASP Insecure Design category in design review checklists to ensure structural security issues are caught before development investment makes them costly to fix.** Insecure design cannot be remediated by patching — it requires architectural change. The best practice is to review application designs against common insecure design patterns (password reset flows, privilege escalation paths, object-level access models, trust assumptions about external inputs) before the design is handed to development. A one-hour design review that catches an insecure design pattern saves weeks of rework after the fact.

Common Mistakes and Misconceptions

Misconception: Treating the OWASP Top 10 as a complete application security standard rather than a prioritised list of the most common critical risks.

Correct approach: The OWASP Top 10 covers the most prevalent and impactful vulnerabilities, not all vulnerabilities. The best practice is to use the Top 10 as a baseline minimum and supplement it with application-specific threat modelling, business logic testing, and controls appropriate to the application's data sensitivity and threat profile.

Misconception: Assuming that an application with no web interface does not need to be assessed against the OWASP Web Top 10.

Correct approach: Many OWASP Web Top 10 categories apply to backend services and APIs as well as browser-facing applications. Injection, cryptographic failures, authentication failures, logging failures, and SSRF all apply regardless of whether the application has a traditional web front end. Assessment scope should be determined by the application's attack surface, not by its interface type.

Misconception: Relying solely on automated scanning tools to identify BOLA and Broken Function Level Authorisation vulnerabilities.

Correct approach: These vulnerabilities require understanding the relationship between authenticated identities and the resources or functions they request — a semantic judgement that automated tools cannot make without application-specific configuration. The best practice is manual test cases specifically designed to verify object-level and function-level authorisation, included in the penetration test scope document.

Misconception: Treating a passed web application penetration test as evidence that the API layer is also secure.

Correct approach: Web application and API security tests have different scopes, test cases, and vulnerability targets. The OWASP API Security Top 10 contains categories — BOLA, Unrestricted Resource Consumption, Improper Inventory Management — that have no direct equivalent in web application testing. The best practice is separate, dedicated API security testing alongside web application testing for any application with an API surface.

table: OWASP Web Top 10 (2021) and API Top 10 (2023) — Security Professional Quick Reference

Use this table as a working reference when scoping application security assessments, reviewing architecture designs, or communicating findings to development teams. Each entry maps to its list category, the core risk, and the primary cloud practitioner action.

Category	List	Core Risk	Practitioner Action
A01: Broken Access Control	Web	Users access data or functions beyond their authorisation	Verify server-side authorisation on every endpoint; test horizontal and vertical escalation
A02: Cryptographic Failures	Web	Sensitive data exposed due to	Review cipher suites, key

		weak/absent encryption	management, TLS enforcement, and encryption at rest
A03: Injection	Web	Attacker-controlled data executed as commands/queries	Verify parameterised queries; include injection in DAST scope
A04: Insecure Design	Web	Structural security flaws in application architecture	Conduct design review before development; use threat modelling
A05: Security Misconfiguration	Web	Default or incorrect configurations expand attack surface	Run CSPM; include misconfiguration in cloud config audits
A06: Vulnerable Components	Web	Known-vulnerable libraries/frameworks in production	Require SCA in CI/CD; maintain SBOM; patch per SLA
A07: Auth Failures	Web	Weak authentication or session management	Enforce MFA; review session expiry, invalidation, and token rotation
A08: Integrity Failures	Web	Unverified code/data updates; supply chain attacks	Sign build artefacts; review CI/CD pipeline security
A09: Logging Failures	Web	Insufficient security event logging and monitoring	Define logging requirements at design; integrate with SIEM
A10: SSRF	Web/Cloud	Server fetches attacker-controlled internal resources	Validate and allowlist URLs; enforce IMDSv2; block metadata IP ranges
API1: BOLA	API	Caller accesses another user's objects via identifier manipulation	Manual BOLA test cases on every endpoint with resource identifiers
API2: Broken Auth	API	Weak token validation or credential stuffing exposure	Review token algorithms, expiry, replay protection, and rate limiting
API3: BOPLA	API	Mass assignment — API applies more properties than intended	Restrict accepted properties explicitly; test

			mass assignment patterns
API4: Unrestricted Consumption	API	No rate limiting enables DoS or financial resource exhaustion	Implement throttling at API gateway and application layer
API5: Broken Function Auth	API	Admin functions accessible to standard users	Test all privileged endpoints for function-level authorisation bypass
API9: Improper Inventory	API	Shadow APIs accumulate without security governance	Maintain API inventory; run API discovery scanning periodically

How to use this: This table covers the highest-priority categories. Complete OWASP lists are available at owasp.org. For CCSP exam purposes, focus on the specific cloud relevance of A05, A10, and API1.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A cloud application allows users to export their data as a CSV by submitting a request to `/export/{userID}`. Penetration testing reveals that an authenticated user can export any other user's data by changing the `userID` in the URL. Which OWASP Web Top 10 category does this vulnerability fall under, and what is the correct remediation?

- A) A. A03: Injection; the remediation is to parameterise the SQL query that retrieves the exported data.
- B) B. A01: Broken Access Control; the remediation is to verify server-side that the authenticated user's identity matches the `userID` in the request before processing the export.
- C) C. A07: Identification and Authentication Failures; the remediation is to require re-authentication before data exports.
- D) D. A10: Server-Side Request Forgery; the remediation is to validate the URL submitted in the export request.

Answer: B. A01: Broken Access Control; the remediation is to verify server-side that the authenticated user's identity matches the `userID` in the request before processing the export.

Q2. The Capital One 2019 breach relied on an attacker making the application server send an HTTP request to the AWS EC2 metadata service at `169.254.169.254`, which returned temporary IAM credentials. Which OWASP Web Top 10 (2021) category does this attack technique represent?

- A) A. A05: Security Misconfiguration, because the metadata service should have been disabled.
- B) B. A03: Injection, because the attacker injected a malicious URL into the application.
- C) C. A10: Server-Side Request Forgery (SSRF), because the application was manipulated into making a server-side HTTP request to an attacker-controlled target.

D) D. A01: Broken Access Control, because the IAM role returned credentials the attacker should not have been able to access.

Answer: C. A10: Server-Side Request Forgery (SSRF), because the application was manipulated into making a server-side HTTP request to an attacker-controlled target.

Q3. Which OWASP API Security Top 10 (2023) category most commonly goes undetected by automated web application scanners and requires specifically designed manual test cases?

A) A. API4: Unrestricted Resource Consumption, because automated tools cannot generate sufficient request volume.

B) B. API1: Broken Object Level Authorisation (BOLA), because it requires understanding the relationship between authenticated identities and the resource identifiers they request.

C) C. API2: Broken Authentication, because authentication testing requires intercepting tokens in transit.

D) D. API10: Unsafe Consumption of Third-Party APIs, because external APIs are out of scope for automated tools.

Answer: B. API1: Broken Object Level Authorisation (BOLA), because it requires understanding the relationship between authenticated identities and the resource identifiers they request.

Q4. A development team proposes to use a popular open-source authentication library that has not released an update in eighteen months. Which OWASP Web Top 10 category is most directly implicated, and what should the Cloud Security Practitioner recommend?

A) A. A07: Identification and Authentication Failures; recommend replacing it with a proprietary authentication solution to eliminate open-source risk.

B) B. A06: Vulnerable and Outdated Components; recommend reviewing the library against current vulnerability databases (NVD, OSV), assessing whether security patches have been backported, and identifying a maintained alternative if the library is no longer actively supported.

C) C. A08: Software and Data Integrity Failures; recommend signing the library with a code signing certificate before incorporating it.

D) D. A04: Insecure Design; recommend redesigning the authentication flow to avoid third-party libraries.

Answer: B. A06: Vulnerable and Outdated Components; recommend reviewing the library against current vulnerability databases (NVD, OSV), assessing whether security patches have been backported, and identifying a maintained alternative if the library is no longer actively supported.

Q5. An organisation's cloud application team has just completed a web application penetration test that found no critical vulnerabilities. The same application also exposes a REST API consumed by mobile clients. What should the Cloud Security Practitioner recommend?

A) A. No further assessment is necessary since the penetration test cleared the application.

B) B. A dedicated API security assessment should be commissioned, specifically scoping for OWASP API Top 10 vulnerabilities including BOLA, Broken Function Level Authorisation, and rate limiting — categories that web application penetration tests do not systematically cover.

C) C. The mobile clients should be tested for certificate pinning bypass, which is the primary API security risk for mobile consumers.

D) D. The API documentation should be reviewed for completeness, and any undocumented endpoints should be removed before further testing.

Answer: B. A dedicated API security assessment should be commissioned, specifically scoping for OWASP API Top 10 vulnerabilities including BOLA, Broken Function Level Authorisation, and rate limiting — categories that web application penetration tests do not systematically cover.

Reflection Questions

1. Which of the OWASP Web Top 10 categories do you encounter most frequently in the cloud applications your organisation operates or reviews? Is there a pattern in which categories appear most often?
2. Consider an API your organisation exposes. Has it been specifically tested for BOLA? If not, what would a BOLA test case for that API look like?
3. The OWASP list added Insecure Design as a separate category in 2021, signalling a shift in how the community thinks about where security failures originate. How would you explain this addition to a development manager who thinks of security primarily as a testing activity?
4. SSRF entered the OWASP Web Top 10 because of its particular relevance to cloud environments. What user-controllable URL inputs or server-side fetch operations exist in applications you are responsible for, and have they been specifically tested for SSRF?
5. How does your organisation currently use OWASP lists in practice? Are they referenced in assessment scope documents, remediation guidance, or risk reporting? Where could they be applied more systematically?

Key Takeaways

- The OWASP Web Application Top 10 (2021) and API Security Top 10 (2023) are operational intelligence tools, not academic references — they identify where attacks actually happen and where defences most often fail.
- Broken Access Control (A01) is the most common web application vulnerability found in practice. It requires architectural remediation — server-side authorisation checks — not patching.
- SSRF (A10) entered the web list specifically because of cloud environments, where internal metadata services provide working credentials to successful SSRF attacks. IMDSv2 enforcement and URL allowlisting are essential mitigations.
- BOLA (API1) is the most prevalent and most commonly missed API security vulnerability. It requires manual test cases designed to verify object-level authorisation, not just automated scanning.
- The OWASP API Security Top 10 addresses a distinct risk profile from the web list. A clean web application penetration test provides no assurance about API security; separate, dedicated API security assessment is required.
- Insecure Design (A04) cannot be remediated by testing or patching — it requires architectural redesign. The only cost-effective prevention is security design review at the architecture phase.
- OWASP category language should be used in security findings, assessment scope documents, and risk reports to establish a shared vocabulary between security professionals and development teams.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Download the OWASP Web Application Top 10 (2021) and API Security Top 10 (2023) documents from owasp.org and review the detailed descriptions of at least three categories you are less familiar with. Map each to a real application in your environment.

7. Review the scope document for the most recent penetration test conducted on a cloud application you are responsible for. Verify that it explicitly covers both OWASP Web Top 10 and OWASP API Top 10 categories if the application has an API surface.
8. Identify one API endpoint in your environment that accepts a resource identifier (user ID, account ID, record ID) and design a manual BOLA test case for it. Execute the test or hand it to your penetration testing team.
9. Check whether your organisation's EC2 instances enforce IMDSv2 at the account or instance level, reducing SSRF exploitability. If not, create a recommendation to enforce it through a service control policy or AWS Config rule.